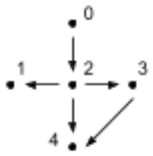
Nombre y apellido: _____

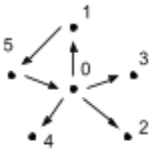
Ejercicios (puntaje)			Nota
1 (4 pts)	2 (3 pts)	3 (3 pts)	

CONSIDERACIONES: RESOLVER CADA EJERCICIO EN UNA HOJA DIFERENTE. LOS EJERCICIOS QUE NO ESTÉN CORRECTOS EN UN 50% NO SUMARÁN PUNTOS PARA LA NOTA FINAL. PARA CADA EJERCICIO DEBE **DEFINIR EL TIPO DE DATO** DE CADA TDA UTILIZADO. **EN TODOS LOS CASOS QUE UTILICE ESTRUCTURAS** QUE NO SEAN TDAs "ESTÁNDAR" **DEBE DEFINIR LOS STRUCTS**.

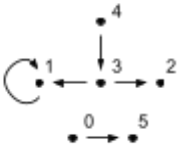
Ejercicio 1: (Implementación a elección) Dado un digrafo, **desarrollar una función que devuelva un 1 si el digrafo es cíclico y 0 si es acíclico**.


is_cyclic = 0


is_cyclic = 0

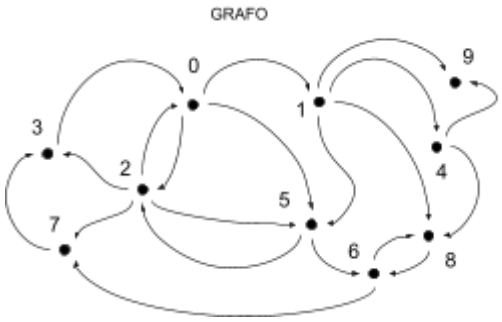

is_cyclic = 1

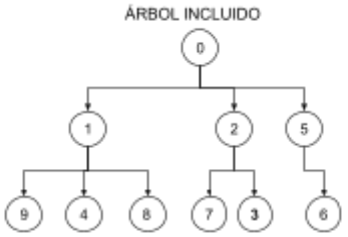

is_cyclic = 0

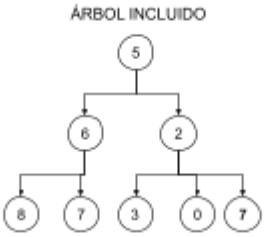

is_cyclic = 1

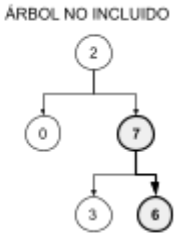
Encabezado función principal: `int` **`is_cyclic`** `(graph* g)`

Ejercicio 2: Desarrollar una función y todas las necesarias para que, dado un árbol n-ario implementado con **TDA list** (para la lista de hijos de cada nodo) y un digrafo implementado con listas de adyacencias (implementadas con **listas dinámicas enlazadas**), **devuelva 1 si el árbol está incluido en el grafo**, y 0 si no lo está. Ejemplo:









Encabezado función principal: `int` **`tree_on_graph`** `(ntn* root, graph_la* g);`

Ejercicio 3: Desarrollar una función y todas las necesarias para **eliminar todos los elementos repetidos de una lista dinámica simplemente enlazada**, recorriendo la lista una sola vez y utilizando como estructura auxiliar un árbol binario de búsqueda. Los elementos de la lista deben ser genéricos y se cuenta con una función **`int cmp (LIST_ELEM a, LIST_ELEM b)`** que devuelve un número **negativo** si **a < b**, un número **positivo** si **a > b** y **0 (cero)** si **a = b**.

Encabezado función principal: `void` **`delete_duplicates`** `(node** h,int` **`cmp`** `(LIST_ELEM a, LIST_ELEM b));`